



Institut supérieur des Études Technologiques de Radès
Département Technologies de l'Informatique

Rapport du Projet d'Intégration



Application Web de Suivi Médical des Patients

Réalisé par :

Mohamed Rayen SAIDI & Mohamed Radhouen KSOURI



Classe : L2-DSII

Année Universitaire : 2025 - 2026

Sommaire

Sommaire	
Liste des Figures	
Liste des Tableaux	
Chapitre 1 : Introduction Générale	1
1.1 Contexte du projet	1
1.2 Objectifs	1
1.3 Étude de l'existant	1
1.4 Solution proposée	1
Chapitre 2 : Analyse des Besoins	2
2.1 Besoins fonctionnels	2
2.1.1 Gestion des utilisateurs	2
2.1.2 Gestion des patients	2
2.1.3 Gestion des consultations	2
2.1.4 Tableau de bord	2
2.1.5 Paramètres	2
2.2 Besoins non fonctionnels	3
2.3 Acteurs du système	3
Chapitre 3 : Méthodologie et Gestion de Projet	4
3.1 Méthodologie Agile Scrum	4
3.1.1 Rôles Scrum	4
3.1.2 Cérémonies Scrum	4
3.1.3 Outils de gestion	4
3.2 Planification prévisionnelle — Diagramme de Gantt	5
3.3 Product Backlog	5
Chapitre 4 : Conception	6
4.1 Architecture technique	6
4.1.1 Stack technique	6
4.1.2 Schéma d'architecture de l'application	7
4.2 Design patterns et patterns architecturaux	8
4.2.1 Pattern Singleton (DataService)	8
4.2.2 Pattern Guard (Protection des routes)	8
4.2.3 Pattern Repository (Isolation par utilisateur)	8
4.2.4 Pattern Intercepteur HTTP	8
4.2.5 Lazy Loading des composants	8
4.3 Modélisation UML	9

4.3.1 Diagramme de cas d'utilisation	9
4.3.2 Diagrammes de séquence	11
4.3.3 Diagramme de classes	17
4.4 Modèle de base de données	18
Chapitre 5 : Réalisation.....	19
5.1 Environnement de développement	19
5.2 Structure de l'application Angular.....	19
5.3 Itération 1 — Sprint 4 (7 mars – 14 avril 2026).....	19
5.3.1 Module Authentification.....	19
5.3.2 Module Gestion des Patients	20
5.3.3 Module Gestion des Consultations	21
5.3.4 Tableau de bord	24
5.4 Itération 2 — Sprint 5 (27 avril 2026).....	24
5.4.1 Gestion des comptes utilisateurs (Admin).....	25
5.4.2 Module Paramètres	26
5.4.3 Système de notifications médicales	26
Chapitre 6 : Tests et Validation	27
6.1 Stratégie de tests.....	27
6.2 Tests fonctionnels — Console Chrome (F12) + Postman.....	27
6.3 Tests de performance — Lighthouse	28
6.4 Tests d'accessibilité	28
6.4.1 Axe DevTools (Extension Chrome)	28
6.4.2 Émulation d'appareils — Chrome DevTools.....	29
6.5 Tableau récapitulatif des tests	30
Conclusion et Perspectives	31
7.1 Bilan du projet.....	31
7.2 Perspectives d'amélioration.....	31
Webographie	32
Documentation officielle.....	32
Outils de test et qualité	32
Méthodologie et conception	32

Liste des Figures

Figure 1 : Schéma d'architecture de l'application.....	7
Figure 2 : Diagramme de cas d'utilisation (itération 1)	9
Figure 3 : Diagramme de cas d'utilisation (itération 2)	9
Figure 4 : Diagramme de cas d'utilisation (itération final)	10
Figure 5 : Diagramme de séquence (authentification - itération 1)	11
Figure 6 : Diagramme de séquence (authentification - itération 2)	12
Figure 7 : Diagramme de séquence système (authentification – itération final)	12
Figure 8 : Diagramme de séquence (Créer une consultation - itération 1)	13
Figure 9 : Diagramme de séquence (Créer une consultation - itération 2)	14
Figure 10 : Diagramme de séquence système (Créer une consultation – itération final)	14
Figure 11 : Diagramme de séquence d'objets (Créer une consultation – itération final).....	15
Figure 12 : Diagramme de séquence (Ajouter un patient - itération 1)	15
Figure 13 : Diagramme de séquence (Ajouter un patient - itération 2)	16
Figure 14 : Diagramme de séquence d'objets (Ajouter un patient – itération final).....	16
Figure 15 : Diagramme de classes (itération 1)	17
Figure 16 : Diagramme de classes (itération final)	18
Figure 17: Interface — Page de connexion.....	20
Figure 18: Interface — Liste des patients avec recherche	20
Figure 19: Interface — Formulaire d'ajout d'un patient.....	21
Figure 20: Interface — Détail d'un patient	21
Figure 21: Interface — Formulaire de nouvelle consultation	22
Figure 22: Interface — Liste des consultations avec filtres.....	23
Figure 23: Interface — Détail d'une consultation avec ordonnance	23
Figure 24: Interface — Tableau de bord avec statistiques et alertes (page web complète).....	24
Figure 25: Interface — Liste des utilisateurs (Admin)	25
Figure 26: Interface — Formulaire d'ajout d'un utilisateur.....	25
Figure 27: paramètres	26
Figure 28: Postman	27
Figure 29: Lighthouse	28
Figure 30: Axe DevTools (Extension Chrome)	29
Figure 31: toggle device toolbar	29

Liste des Tableaux

Tableau 1 : Besoins non fonctionnels	3
Tableau 2 : Rôles Scrum	4
Tableau 3 : Planification prévisionnelle (Diagramme de Gantt)	5
Tableau 4 : Product Backlog.....	5
Tableau 5 : Stack technique de l'application.....	6
Tableau 6 : Entités de la base de données.....	18
Tableau 7 : Environnement de développement.....	19
Tableau 8: Tableau récapitulatif des tests	30

Chapitre 1 : Introduction Générale

1.1 Contexte du projet

Dans le domaine médical, la gestion des dossiers patients est souvent réalisée sous format papier ou via des systèmes partiels, ce qui entraîne des pertes d'informations, un manque d'organisation et des difficultés dans le suivi médical. Ce constat est particulièrement vrai dans les cliniques privées et cabinets médicaux en Tunisie, où la transition numérique reste limitée.

Dans ce contexte, le présent projet a pour objectif de concevoir et développer MediTrack, une application web de suivi médical des patients.

1.2 Objectifs

- Digitaliser les dossiers médicaux des patients
- Améliorer l'organisation des consultations
- Centraliser les données des patients pour chaque médecin
- Offrir un tableau de bord avec statistiques et alertes médicales
- Gérer les comptes utilisateurs (médecins et administrateurs)

1.3 Étude de l'existant

En Tunisie, quelques solutions numériques médicales existent, notamment DabaDoc qui propose la prise de rendez-vous en ligne. Cependant, ces plateformes sont orientées vers la gestion des rendez-vous et non vers la gestion complète du dossier patient. Elles ne proposent pas de module d'intelligence artificielle pour l'aide à la décision.

MediTrack se distingue par une gestion complète du dossier patient, une architecture web moderne, une isolation des données par médecin, et un module de notifications médicales automatisées.

1.4 Solution proposée

MediTrack est une Single-Page Application (SPA) développée avec Angular 18 en frontend et PHP/MySQL en backend. Elle permet à chaque médecin de gérer ses propres patients et consultations de manière sécurisée, avec un tableau de bord réactif basé sur Angular Signals.

Chapitre 2 : Analyse des Besoins

2.1 Besoins fonctionnels

2.1.1 Gestion des utilisateurs

- Authentification sécurisée (email + mot de passe hashé avec bcrypt)
- Gestion des rôles : Administrateur / Médecin
- Création, modification et suppression de comptes utilisateurs (admin)
- Modification de son propre profil

2.1.2 Gestion des patients

- Ajouter un nouveau patient avec ses informations médicales complètes
- Modifier les informations d'un patient existant
- Supprimer un patient et toutes ses consultations associées (CASCADE)
- Rechercher un patient par nom, prénom ou téléphone
- Consulter l'historique médical complet d'un patient
- Détection des doublons par téléphone par médecin

2.1.3 Gestion des consultations

- Enregistrer une nouvelle consultation (motif, tension, pouls, température, poids)
- Ajouter diagnostic, observations et ordonnance
- Calcul automatique du montant selon le statut du patient (régulier / non régulier)
- Filtrer les consultations par type, statut et période (date range picker)
- Consulter le détail complet d'une consultation avec ordonnance
- Joindre des photos d'analyses à une consultation

2.1.4 Tableau de bord

- Afficher le nombre total de patients et les répartitions (chronique / non chronique)
- Afficher les statistiques de revenus (régulier / non régulier / total mensuel)
- Afficher les 5 dernières consultations
- Afficher les alertes médicales : patients chroniques sans consultation depuis plus de 3 mois

2.1.5 Paramètres

- Configurer les tarifs de consultation pour les patients réguliers et non réguliers

2.2 Besoins non fonctionnels

Critère	Description
Sécurité	Authentification par session PHP + localStorage ; accès filtré par user_id ; mots de passe hashés (bcrypt) ; guards Angular (authGuard, adminGuard)
Performance	Architecture SPA avec lazy loading des composants ; Angular Signals pour la réactivité sans re-render inutile ; chargement initial unique des données (DataService singleton)
Fiabilité	Base de données MySQL avec contraintes de clés étrangères et CASCADE ; gestion des erreurs côté backend (try/catch PDO)
Ergonomie	Interface simple et intuitive avec sidebar de navigation, topbar et composants réutilisables ; design responsive
Compatibilité	Fonctionne sur navigateurs modernes (Chrome, Firefox, Edge) ; application testée avec Chrome DevTools device emulation
Isolation des données	Chaque médecin ne voit que ses propres patients et consultations grâce au filtrage par user_id côté backend

Tableau 1 : Besoins non fonctionnels

2.3 Acteurs du système

Administrateur

Gère les comptes utilisateurs (CRUD), attribue les rôles, configure les paramètres.

Médecin

Gère ses patients et ses consultations, accède au tableau de bord, consulte l'historique médical.

Chapitre 3 : Méthodologie et Gestion de Projet

3.1 Méthodologie Agile Scrum

Le projet MediTrack a été développé en adoptant la méthodologie Agile Scrum. Ce choix s'est imposé naturellement face aux exigences du projet pédagogique : la nécessité d'itérations régulières validées en classe, la capacité d'intégrer les retours du professeur à chaque séance, et une planification flexible permettant d'ajuster les priorités en cours de semestre.

3.1.1 Rôles Scrum

Rôle Scrum	Membre	Responsabilités	Outils
Product Owner	Rayen	Définit les User Stories, priorise le backlog	Notion
Scrum Master	Radhouene	Facilite les cérémonies Scrum, lève les obstacles	Discord
Dev Frontend	Équipe	Angular 18, composants, UI, Signals	VS Code / GitHub
Dev Backend	Équipe	PHP, MySQL, API REST	XAMPP / phpMyAdmin

Tableau 2 : Rôles Scrum

3.1.2 Cérémonies Scrum

- Daily Scrum (15 min/séance) : Qu'ai-je fait ? Que vais-je faire ? Obstacles ?
- Sprint Review (fin de sprint) : Démonstration des fonctionnalités au professeur
- Sprint Retrospective : Analyse des améliorations à apporter
- Sprint Planning : Sélection des User Stories selon la vélocité de l'équipe

3.1.3 Outils de gestion

- GitHub : versioning du code source avec commits réguliers par fonctionnalité
- Discord : communication entre les membres de l'équipe
- VS Code : éditeur de code avec extensions Angular et PHP

3.2 Planification prévisionnelle — Diagramme de Gantt

Sprint	Tâches	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12
Sprint 1	Analyse des besoins + diagramme cas utilisation	X	X										
Sprint 2	Diagrammes de séquences + première maquette			X	X								
Sprint 3	Maquettes + diagramme de classes + diagramme relationnel					X	X						
Sprint 4	Première Itération							X	X				
Sprint 5	Deuxième Itération									X	X		
Sprint 6	Tests et corrections											X	X

Tableau 3 : Planification prévisionnelle (Diagramme de Gantt)

3.3 Product Backlog

Le Product Backlog liste toutes les fonctionnalités souhaitées pour MediTrack, classées par priorité selon la formule : **Priorité** = (*Valeur métier* × 2) + *Urgence* - *Complexité*.

ID	User Story	Rôle	Priorité	Complexité	Sprint	Statut
US-01	Se connecter à l'application	Médecin/Admin	Haute	3 pts	S1	Fait
US-02	Consulter la liste des patients	Médecin	Haute	3 pts	S1	Fait
US-03	Ajouter un nouveau patient	Médecin	Haute	5 pts	S1	Fait
US-04	Modifier un patient existant	Médecin	Haute	3 pts	S1	Fait
US-05	Supprimer un patient	Médecin	Moyenne	2 pts	S1	Fait
US-06	Enregistrer une consultation	Médecin	Haute	8 pts	S1	Fait
US-07	Voir le détail d'une consultation	Médecin	Haute	5 pts	S1	Fait
US-08	Filtrer les consultations (date range)	Médecin	Moyenne	8 pts	S1	Fait
US-09	Tableau de bord + statistiques	Médecin/Admin	Haute	8 pts	S1	Fait
US-10	Configurer les tarifs	Admin	Moyenne	3 pts	S1	Fait
US-11	Gérer les comptes utilisateurs	Admin	Haute	13 pts	S2	Fait
US-12	Historique complet d'un patient	Médecin	Moyenne	5 pts	S2	Fait

Tableau 4 : Product Backlog

Chapitre 4 : Conception

4.1 Architecture technique

MediTrack repose sur une architecture multicouche séparant clairement le frontend, le backend et la base de données. Cette séparation permet l'indépendance des couches, la maintenabilité du code, et une communication standardisée via API REST.

4.1.1 Stack technique

Couche	Technologie	Rôle
Frontend	Angular 18 + TypeScript	SPA réactive, routing, composants standalone
UI / Style	CSS personnalisé + Angular Material 18	Composants UI, date picker, mise en page responsive
State Management	Angular Signals (computed, signal)	Réactivité fine, remplacement de NgRx pour ce projet
Backend	PHP 8 (natif, sans framework)	API REST, gestion des sessions, logique métier
Base de données	MySQL (via XAMPP)	Stockage persistant, clés étrangères, intégrité référentielle
Accès DB	PDO (PHP Data Objects)	Requêtes préparées, protection contre les injections SQL
Serveur local	XAMPP (Apache + MySQL)	Environnement de développement local
Versioning	Git + GitHub	Gestion du code source et collaboration

Tableau 5 : Stack technique de l'application

4.1.2 Schéma d'architecture de l'application

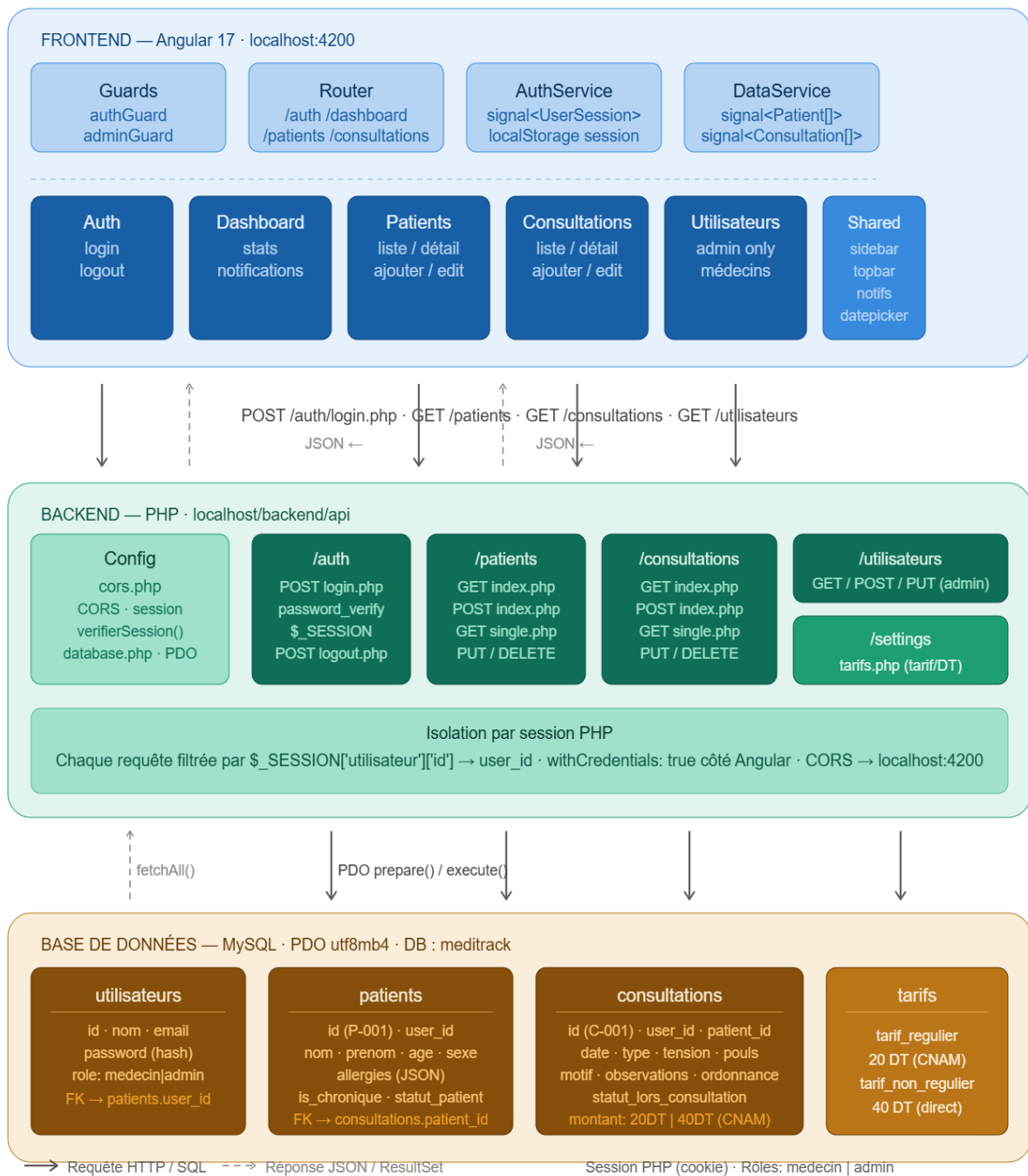


Figure 1 : Schéma d'architecture de l'application

4.2 Design patterns et patterns architecturaux

4.2.1 Pattern Singleton (DataService)

Le DataService est déclaré avec `@Injectable({ providedIn: "root" })`, ce qui fait d'Angular un singleton global pour toute l'application. Ce service est le cœur de l'état de l'application : il charge les données une seule fois au démarrage, les stocke dans des **Signals Angular**, et expose des données calculées (computed signals) à tous les composants qui en ont besoin.

Ce choix architectural garantit une source unique de vérité et évite les incohérences entre composants. Toute mutation (ajout, modification, suppression d'un patient ou d'une consultation) est répercutée immédiatement sur l'ensemble de l'interface sans rechargement de page.

4.2.2 Pattern Guard (Protection des routes)

Deux guards Angular protègent les routes de l'application :

- **AuthGuard** : vérifie que l'utilisateur est connecté (localStorage + Signal). Redirige vers /auth si non authentifié.
- **AdminGuard** : vérifie que l'utilisateur connecté possède le rôle "admin". Redirige vers /dashboard si l'accès est refusé. Protège exclusivement la route /utilisateurs.

4.2.3 Pattern Repository (Isolation par utilisateur)

Côté backend PHP, chaque requête SQL filtre systématiquement les données par le **"user_id"** de la session en cours. Ce principe d'isolation garantit qu'un médecin ne peut jamais accéder aux données d'un autre médecin, même en manipulant les appels API directement.

4.2.4 Pattern Intercepteur HTTP

Un intercepteur Angular (auth.interceptor.ts) est utilisé pour ajouter automatiquement les cookies de session (`{ withCredentials: true }`) à chaque requête HTTP sortante, sans avoir à le préciser dans chaque appel de service.

4.2.5 Lazy Loading des composants

Tous les composants Angular de l'application sont déclarés en standalone et chargés en lazy loading via le routing (loadComponent). Cette approche réduit le bundle initial de l'application et améliore les performances au premier chargement.

4.3 Modélisation UML

4.3.1 Diagramme de cas d'utilisation

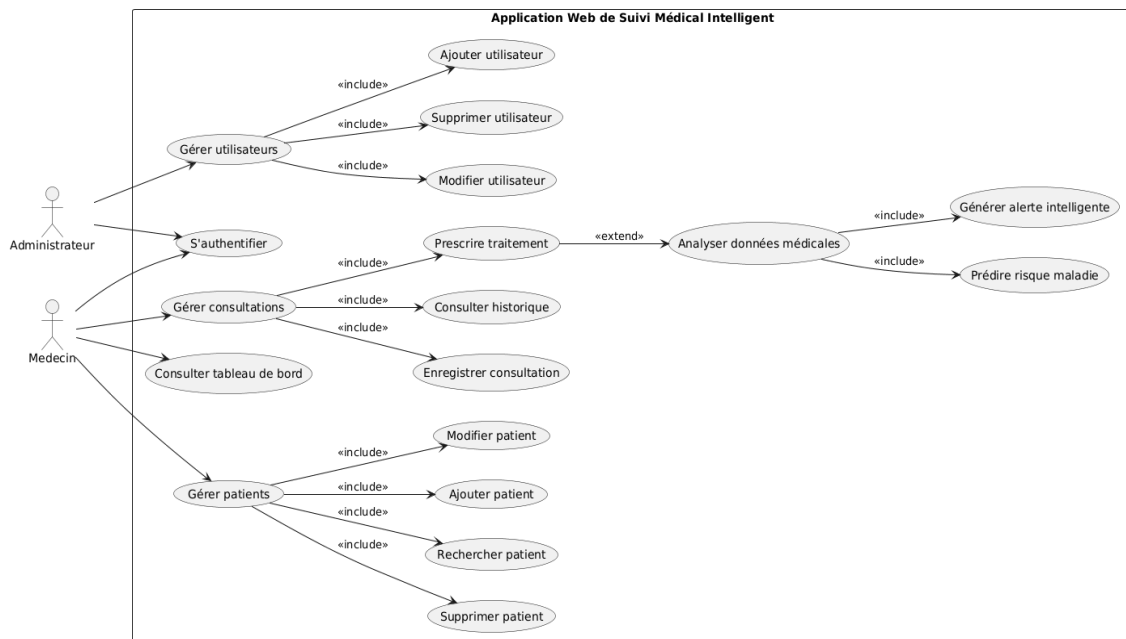


Figure 2 : Diagramme de cas d'utilisation (itération 1)

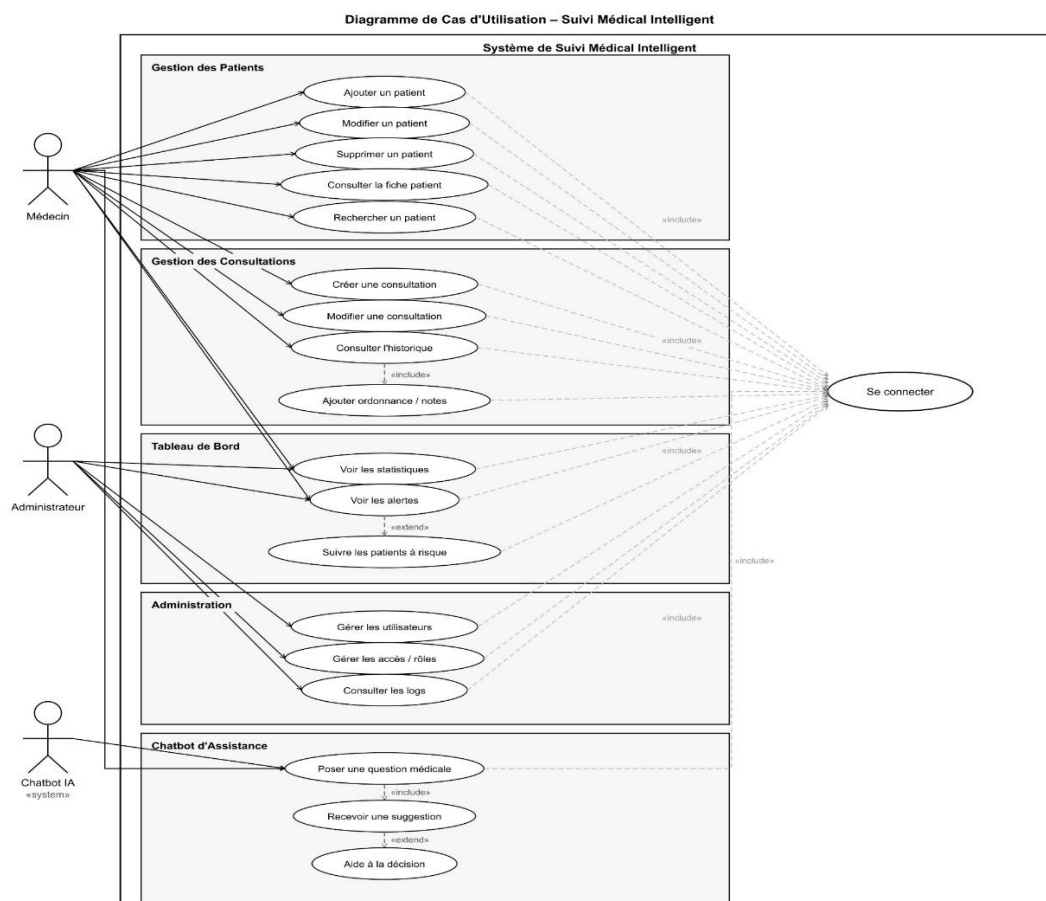


Figure 3 : Diagramme de cas d'utilisation (itération 2)

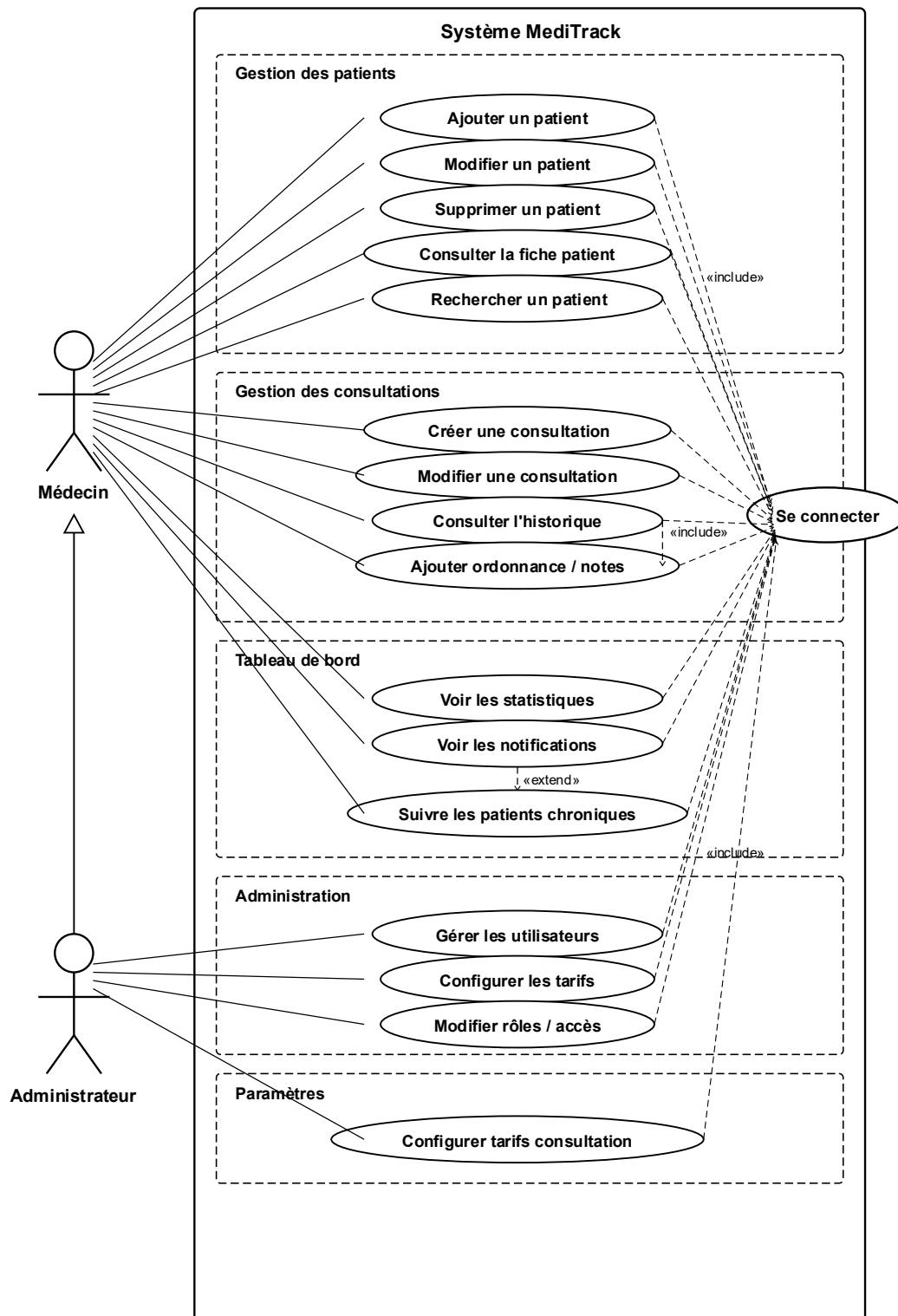


Figure 4 : Diagramme de cas d'utilisation (itération final)

4.3.2 Diagrammes de séquence

Les diagrammes de séquence ci-dessous illustrent les interactions entre les acteurs et le système pour les cas d'utilisation principaux.

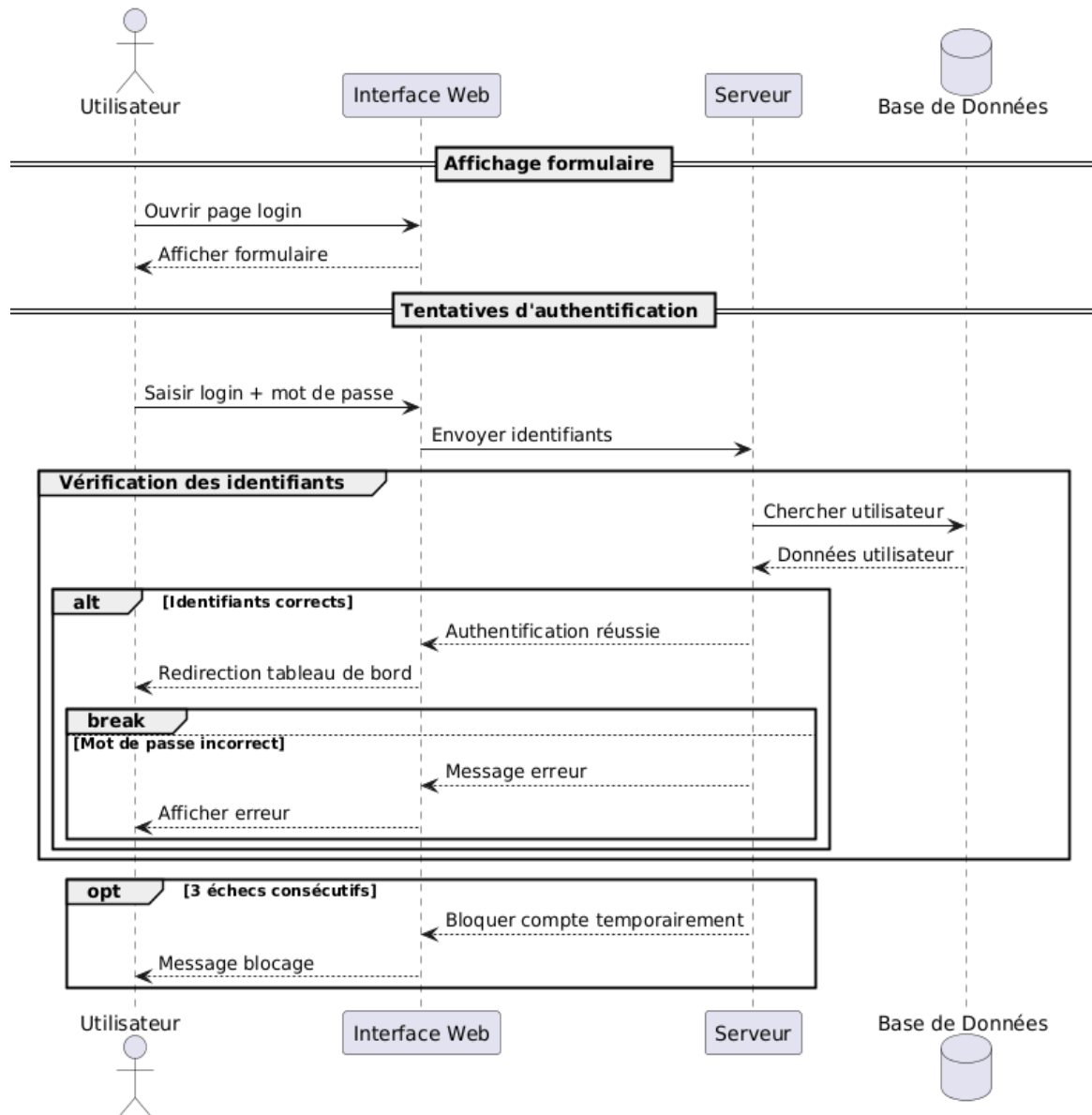


Figure 5 : Diagramme de séquence (authentification - itération 1)

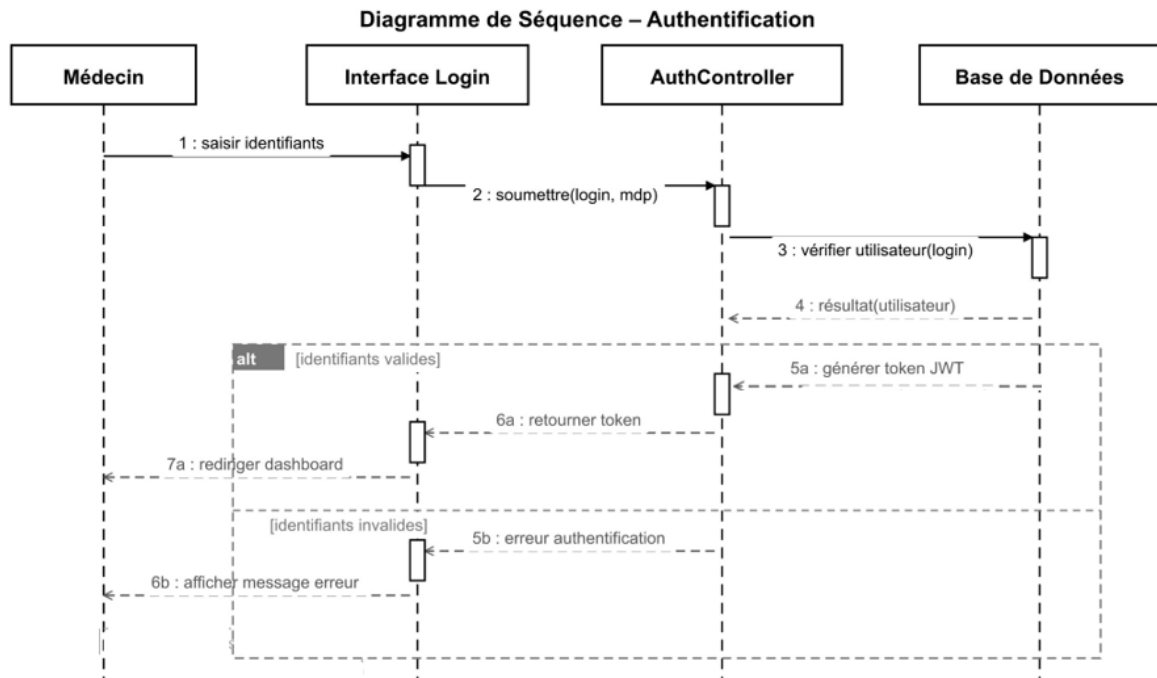


Figure 6 : Diagramme de séquence (authentification - itération 2)

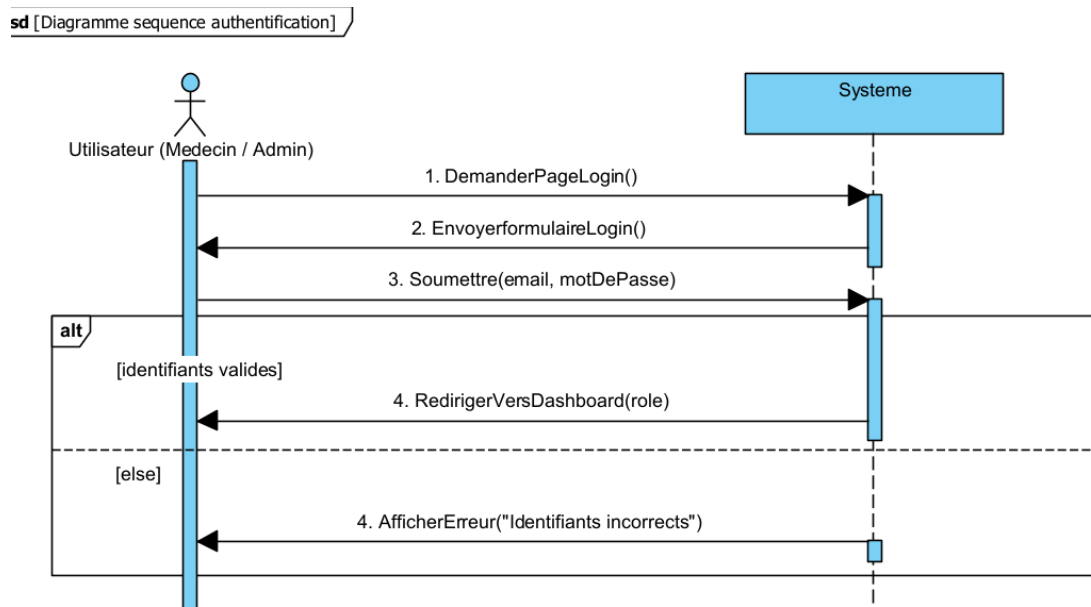


Figure 7 : Diagramme de séquence système (authentification – itération final)

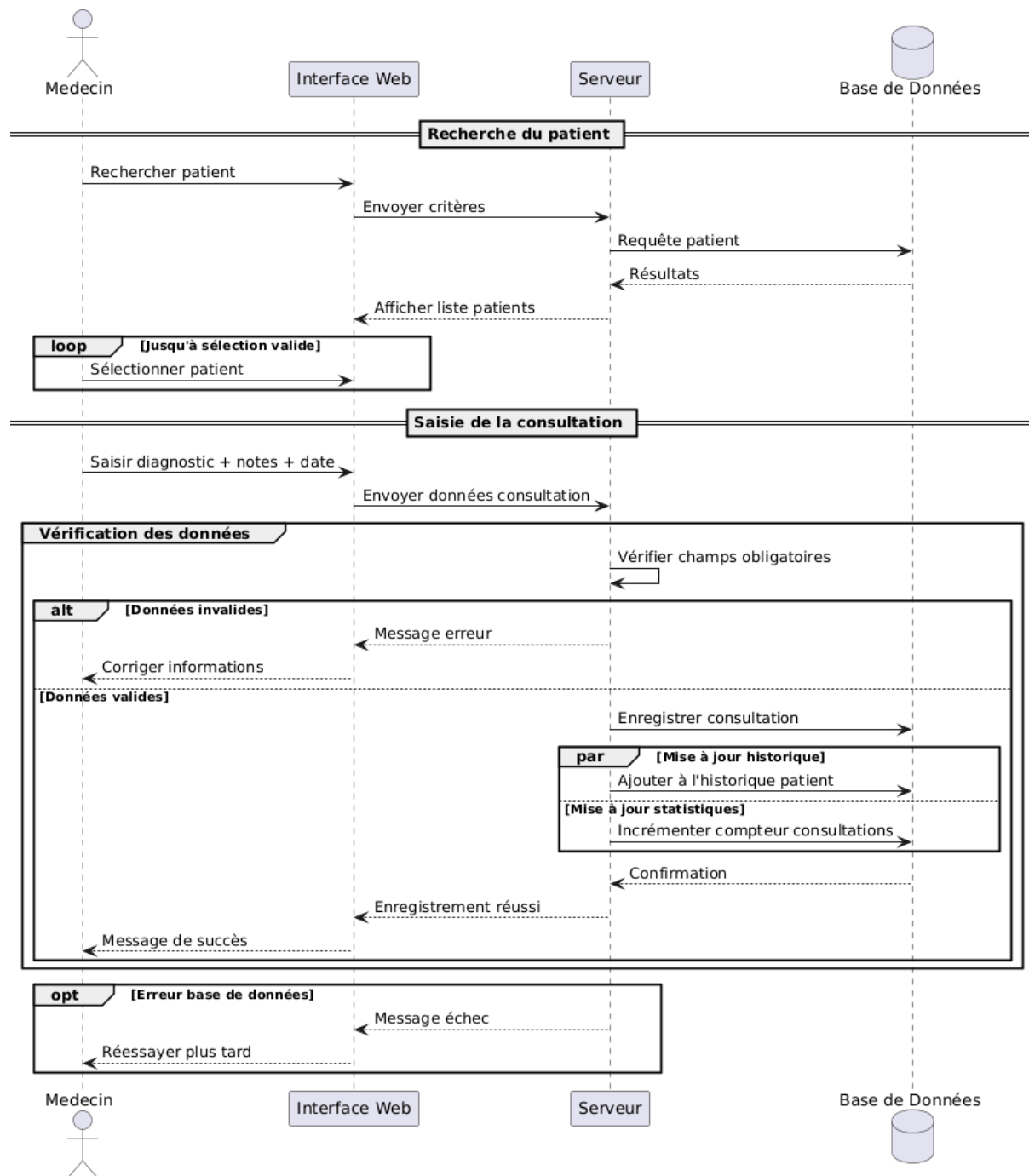


Figure 8 : Diagramme de séquence (Créer une consultation - itération 1)

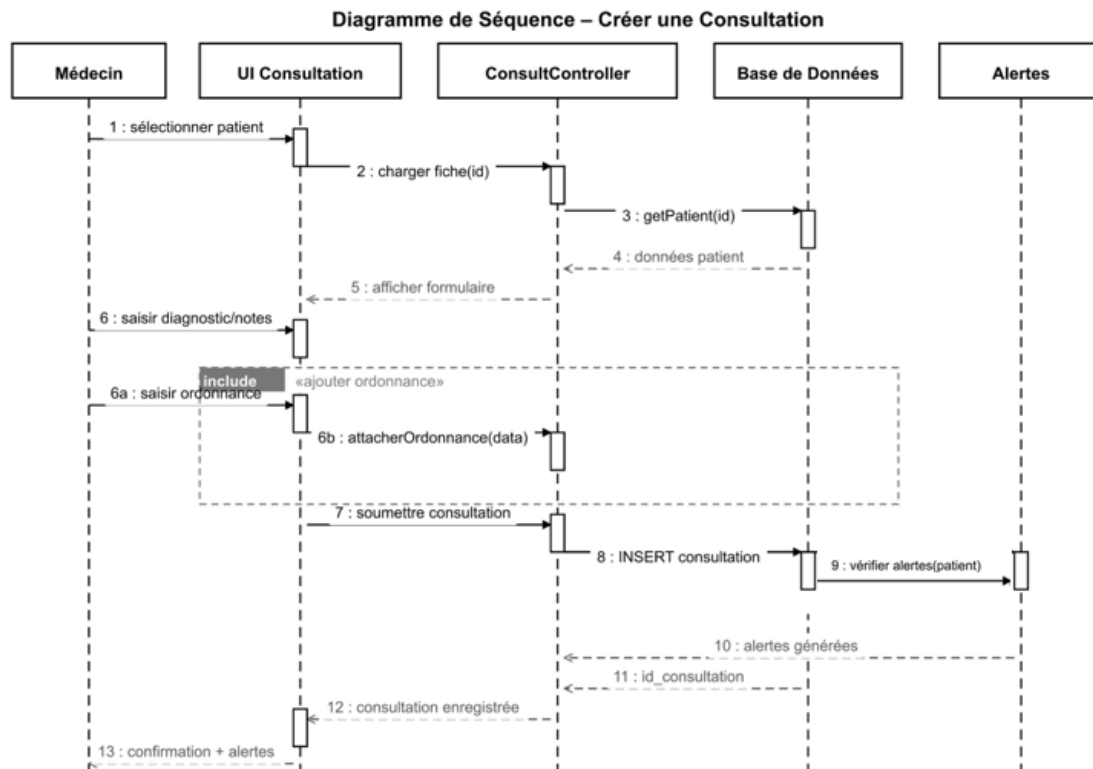


Figure 9 : Diagramme de séquence (Créer une consultation - itération 2)

sd [Diagramme sequence systeme - Créer une Consultation]

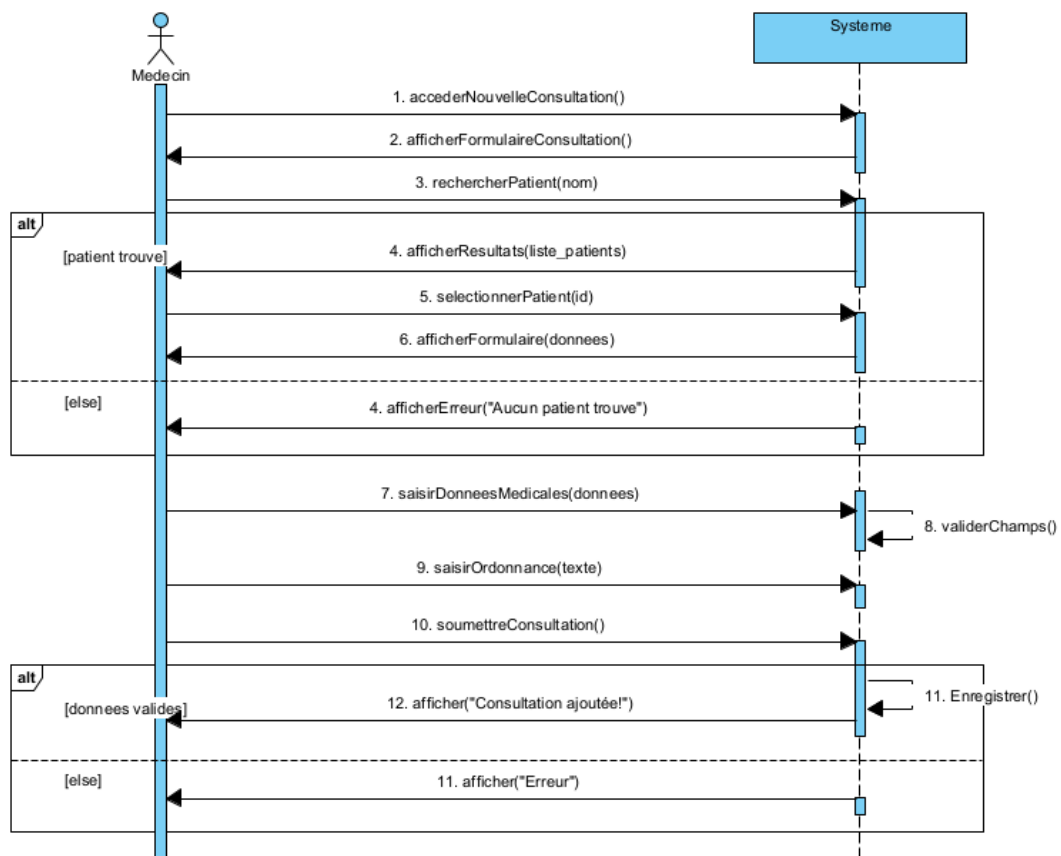


Figure 10 : Diagramme de séquence système (Créer une consultation – itération final)

sd [Diagramme sequence objets - creer une consultation]

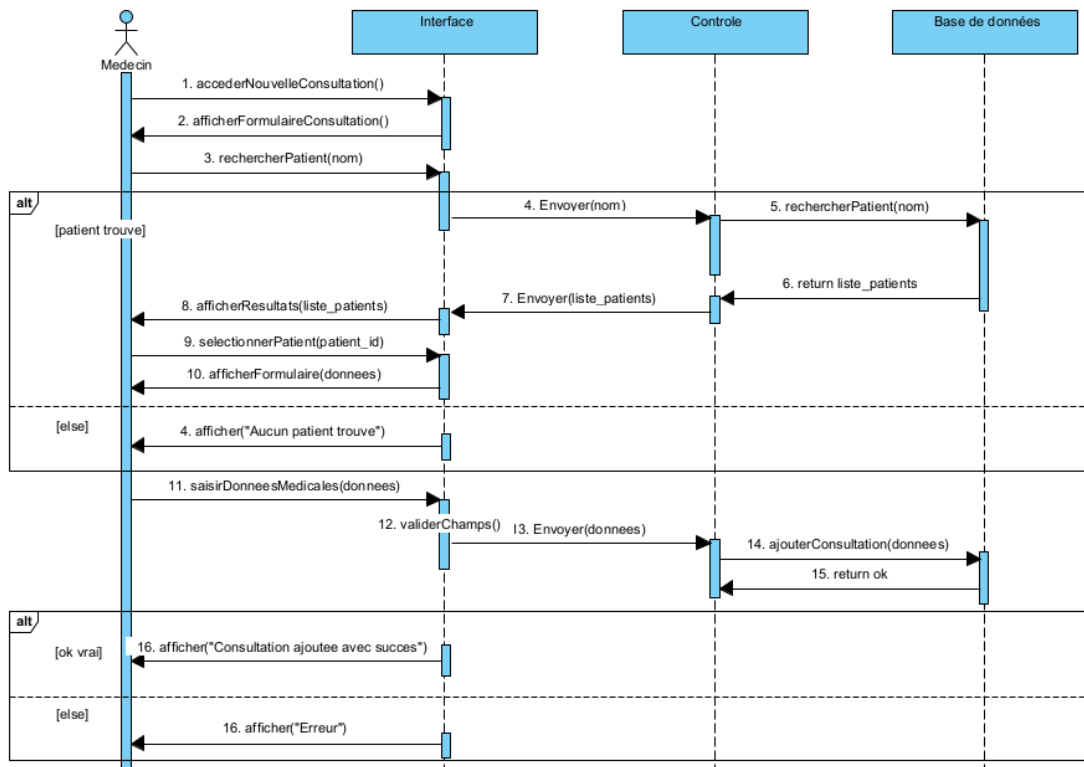


Figure 11 : Diagramme de séquence d'objets (Créer une consultation – itération final)

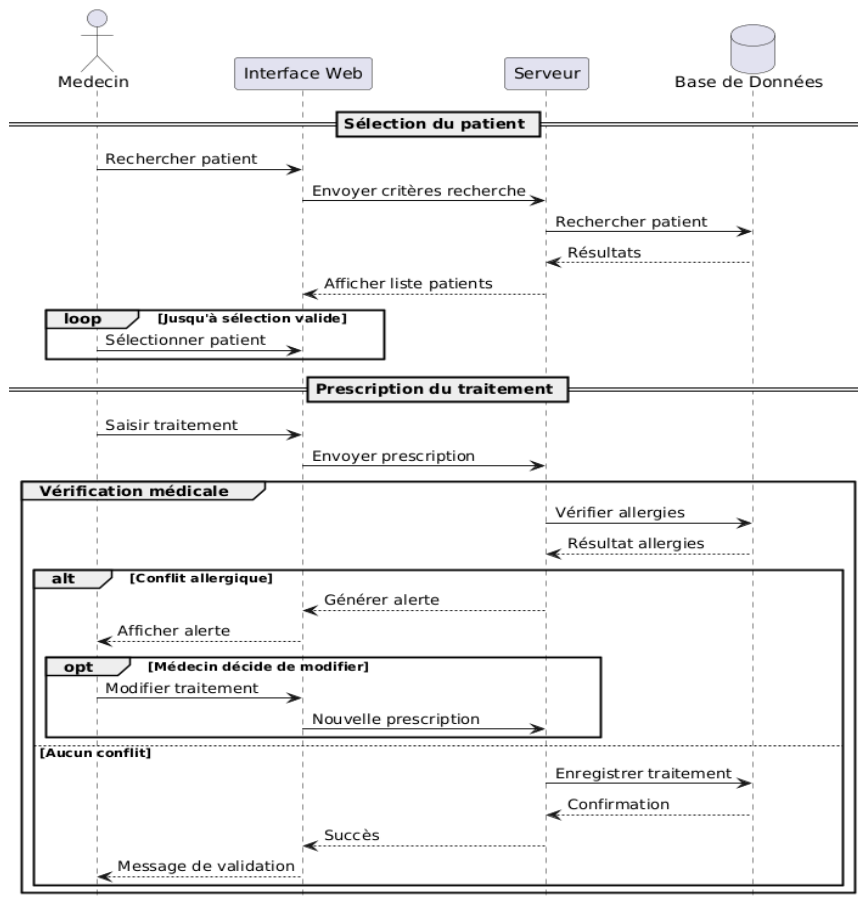


Figure 12 : Diagramme de séquence (Ajouter un patient - itération 1)

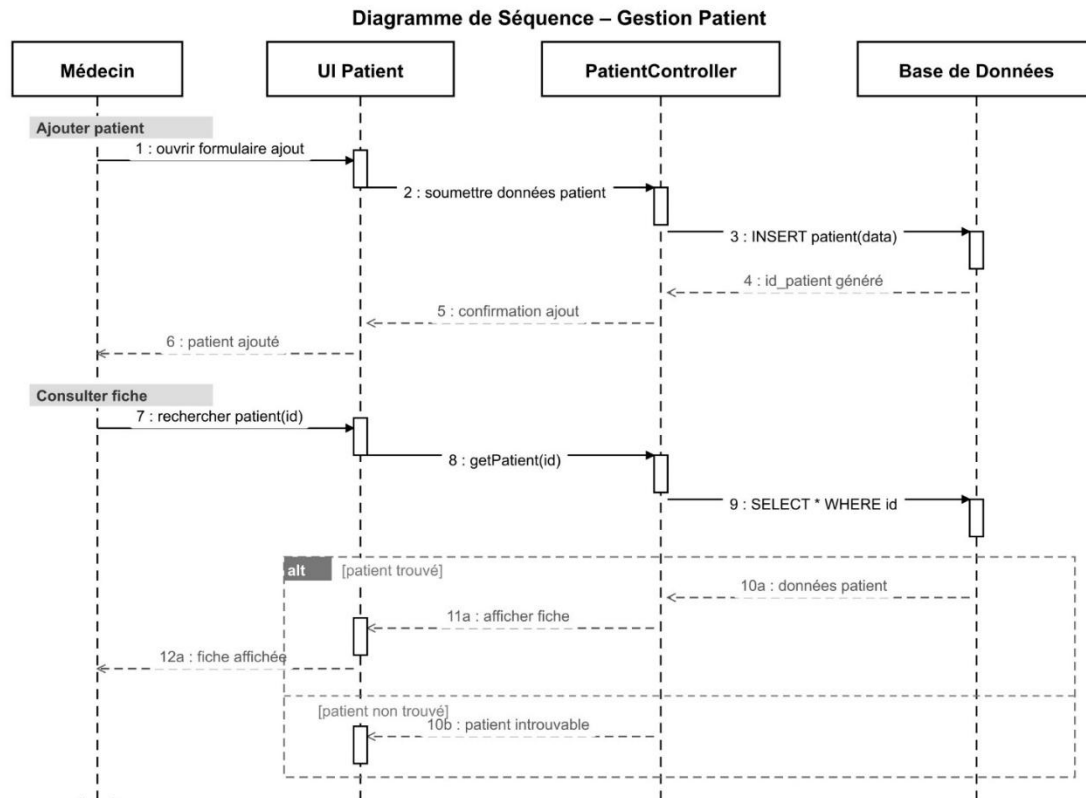


Figure 13 : Diagramme de séquence (Ajouter un patient - itération 2)

sd [Diagramme sequence objets - Ajouter un Patient]

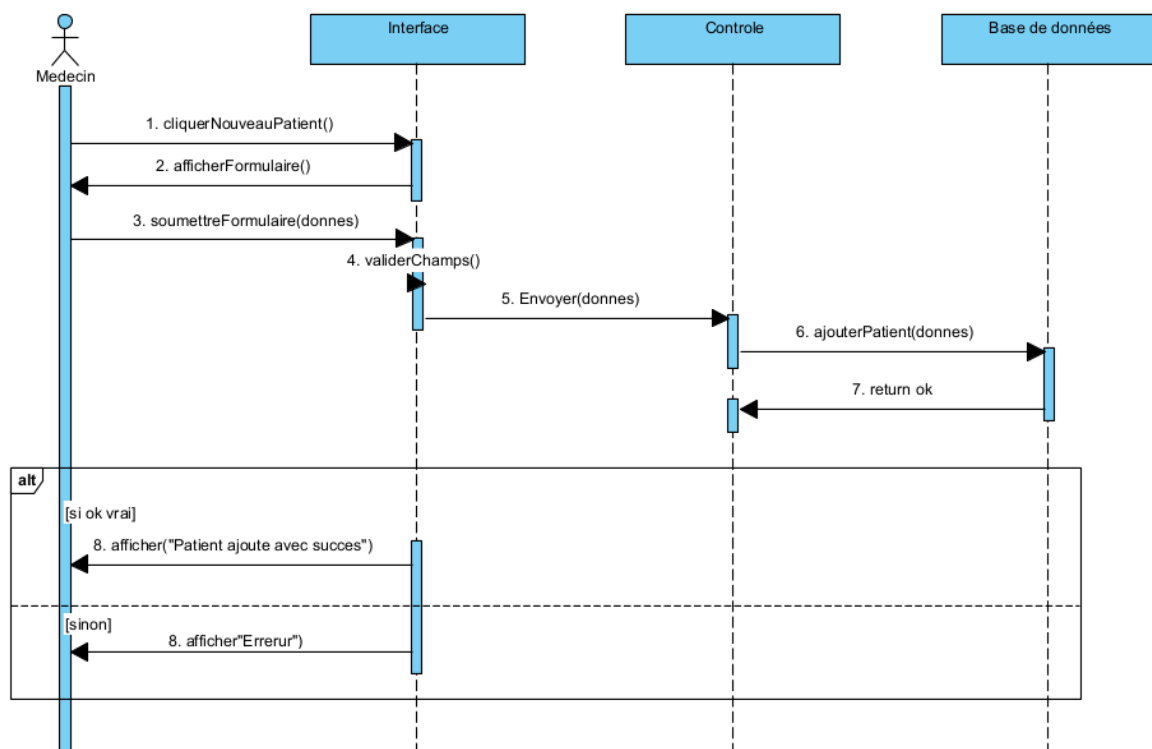


Figure 14 : Diagramme de séquence d'objets (Ajouter un patient – itération final)

4.3.3 Diagramme de classes

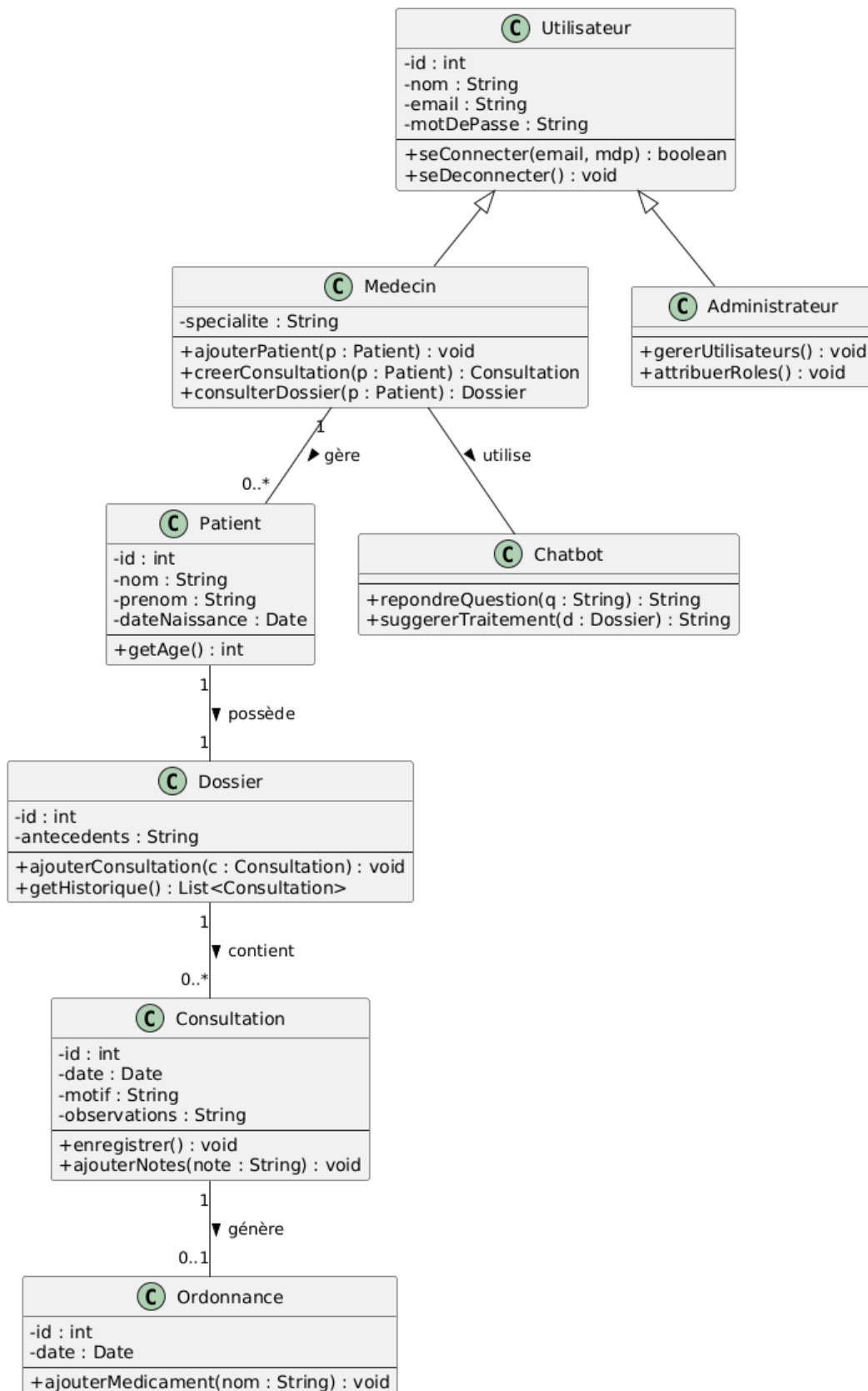


Figure 15 : Diagramme de classes (itération 1)

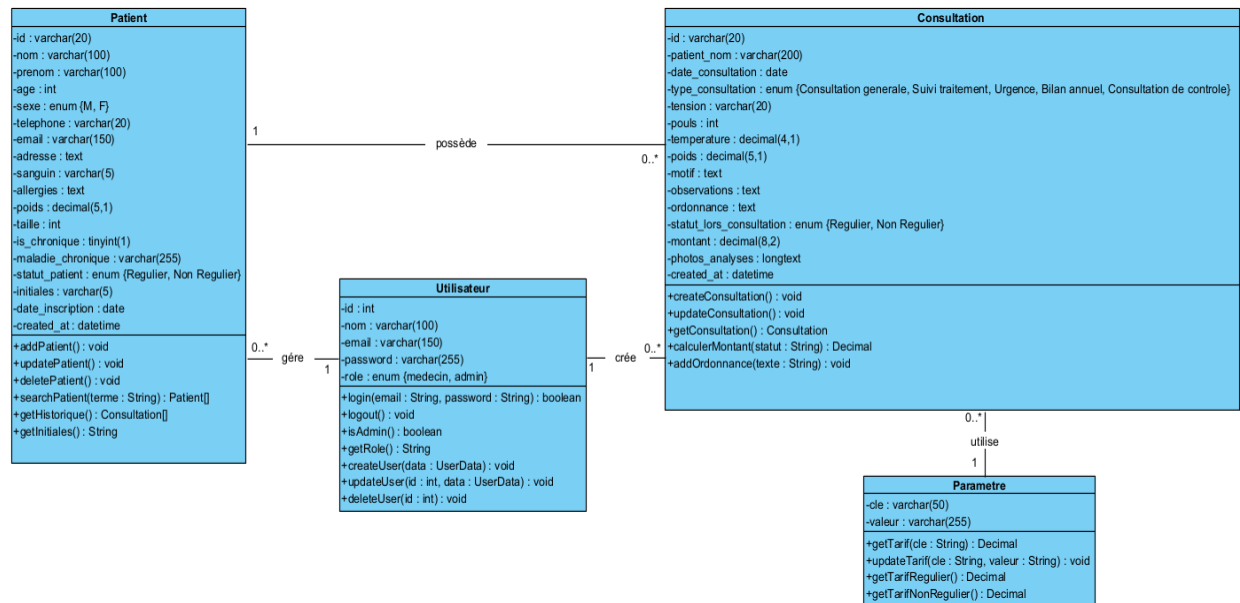


Figure 16 : Diagramme de classes (itération final)

4.4 Modèle de base de données

La base de données de l'application est composée de 4 entités principales :

Entité	Description	Clé étrangère
Utilisateur	Médecins et administrateurs de l'application	—
Patient	Informations médicales et personnelles des patients	user_id → Utilisateur (id)
Consultation	Historique des consultations avec ordonnance et montant	patient_id, user_id
Paramètres	Configuration système (tarifs régulier / non régulier)	—

Tableau 6 : Entités de la base de données

Chapitre 5 : Réalisation

5.1 Environnement de développement

Outil / Logiciel	Utilisation
XAMPP (Apache + MySQL)	Serveur web local et base de données MySQL
Visual Studio Code	Éditeur principal (extensions Angular, PHP)
Node.js + Angular CLI 18	Compilation et développement Angular (ng serve / ng build)
phpMyAdmin	Administration visuelle de la base de données MySQL
Postman	Test manuel des endpoints API REST PHP
Git + GitHub	Gestion du versioning et sauvegarde du code source
Chrome DevTools (F12)	Débogage frontend, onglets Console, Network, Application
Figma	Création des maquettes et wireframes d'interfaces

Tableau 7 : Environnement de développement

5.2 Structure de l'application Angular

L'application Angular suit une architecture modulaire en features, conforme aux bonnes pratiques Angular 18 :

- Services partagés, guards, intercepteurs, modèles
- Modules fonctionnels isolés
- Composants réutilisables, pipes, modèles

5.3 Itération 1 — Sprint 4 (7 mars – 14 avril 2026)

La première itération a permis d'implémenter le cœur fonctionnel de l'application. Les modules suivants ont été réalisés :

5.3.1 Module Authentification

Le module d'authentification gère la connexion et la déconnexion sécurisée. Le backend vérifie les identifiants via **PDO** et crée une session **PHP** côté serveur. Le frontend stocke les informations de l'utilisateur dans un Signal Angular et dans le **localStorage** pour la persistance entre rechargements.

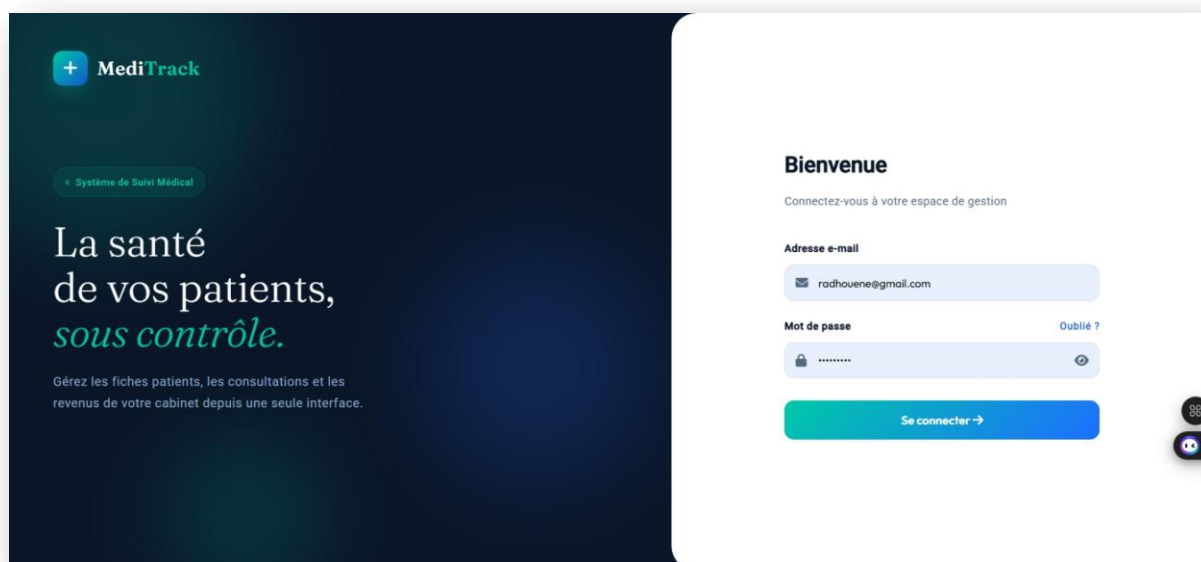


Figure 17: Interface — Page de connexion

5.3.2 Module Gestion des Patients

Le module de gestion des patients permet la gestion complète des dossiers patients. Chaque patient possède un identifiant unique généré côté PHP, des initiales colorées générées automatiquement, et un indicateur de statut (Régulier/Non Régulier).

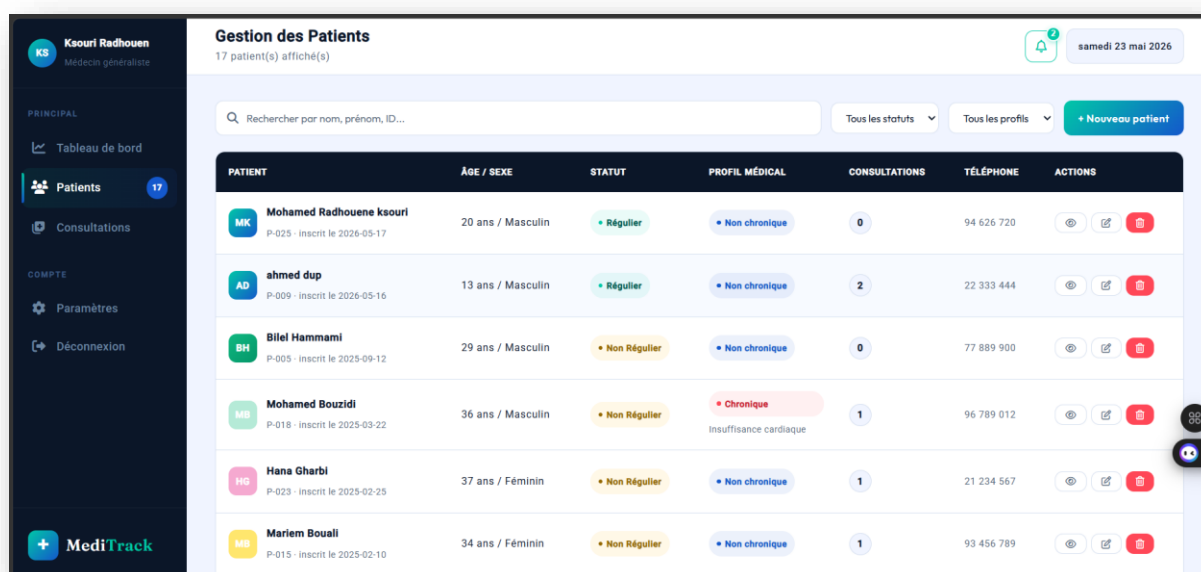


Figure 18: Interface — Liste des patients avec recherche

Kaouri Radhouen
Médecin généraliste

Nouveau Patient
Remplissez les informations de la fiche patient

Informations générales

PRÉNOM * : Foulén

NOM * : Foulén

ÂGE * : Ex: 45

SEXE : Masculin

TÉLÉPHONE * : Ex: 06 12 34 56 78

EMAIL : patient@email.com

GROUPE SANGUIN : A+

STATUT PATIENT * : Régulier

ADRESSE : Adresse complète

ALLERGIES CONNUES

Figure 19: Interface — Formulaire d'ajout d'un patient

Kaouri Radhouen
Médecin généraliste

Fatima Ben Ali
P-013 - 58 ans - Femelle

Informations personnelles

Statut : Régulier

Groupe sanguin : O+

Téléphone : 91 234 567

Email : fatima.benali@gmail.com

Inscrit le : 2025-01-15

Consultations : 3

Revenu généré : 75 DT

PROFIL MÉDICAL

Chronique : Hypertension

MESURES

Poids : 68.0 kg

Taille : 162 cm

Historique des consultations

Date	Consultation	Statut	Revenu
2025-05-22	Consultation générale	Régulier - 25 DT	25 DT
2025-05-19	Consultation générale	Régulier - 25 DT	25 DT
2025-04-28	Consultation générale	Régulier - 25 DT	25 DT

[Modifier la fiche](#)

[Nouvelle consultation](#)

Figure 20: Interface — Détail d'un patient

5.3.3 Module Gestion des Consultations

Le module de consultations gère l'enregistrement des consultations médicales avec calcul automatique du montant, affichage de l'ordonnance, et possibilité de joindre des photos d'analyses. Un composant personnalisé permet le filtrage par période.

KS

Kaouri Radhouen

Médecin généraliste

PRINCIPAL

Tableau de bord

Patients 17

Consultations

COMPTE

Paramètres

Déconnexion

MediTrack

Nouvelle Consultation

Enregistrer une consultation patient

samedi 23 mai 2026

Nouvelle consultation

← Retour

Les champs marqués * sont obligatoires

Informations générales

PATIENT *
Bilel Hammami (P-005) — Non Régulier

DATE *
23/05/2026

TYPE DE CONSULTATION
Consultation générale

Constantes vitales

TENSION ARTÉRIELLE (MMHG)
ex: 16/5/105

POULS (BPM)
0

TEMPÉRATURE (°C)
37

POIDS (KG)
0

Notes cliniques

MOTIF DE CONSULTATION *
Décrivez le motif...

OBSERVATIONS CLINIQUES
Examen clinique, symptômes...

ORDONNANCE / TRAITEMENT PRESCRIT
Médicaments, dosages, durée...

Photos d'analyses

OPTIONNEL

Cliquez pour ajouter des photos
Résultats d'analyses, radios, ECG... (JPG, PNG, PDF)

Facturation

STATUT LORS DE CETTE CONSULTATION
Non Régulier

TARIF APPLIQUÉ
Patient Non Régulier → 40 DT (payé directement)

MONTANT À ENCAISSER
40 DT

Annuler

Enregistrer la consultation

Figure 21: Interface — Formulaire de nouvelle consultation

The screenshot displays the 'Consultations' section of the MediTrack application. The left sidebar shows the user 'Ksouri Radhouen' and navigation options like 'Tableau de bord', 'Patients', 'Consultations', 'Paramètres', and 'Déconnexion'. The main area shows a list of consultations with columns for DATE, PATIENT, TYPE, CONSTANTES, and ACTIONS. A calendar overlay for May 2026 is visible, showing dates from 1 to 31. The consultations list includes:

DATE	PATIENT	TYPE	CONSTANTES	STATUT	ACTIONS
2026-05-22	Fatima Ben Ali C-027	Consultation générale	0 bpm 37°C		
2026-05-22	ahmed dup C-026	Consultation générale	0 bpm 37°C		
2026-05-17	ahmed dup C-025	Consultation de contrôle	0 bpm 37°C		
2026-05-16	ahmed dup C-009	Consultation générale	0 bpm 37°C	Régulier	25 DT
2026-05-12	Mariam Bouali	Consultation générale	120/78 68 bpm	Non Régulier	40 DT

Figure 22: Interface — Liste des consultations avec filtres

The screenshot displays the 'Details' view of a consultation for patient 'Tarek Triki' (P-022, 49 ans, Masculin). The left sidebar is the same as in Figure 22. The main area is divided into several sections:

- Header:** Patient name 'Tarek Triki' and a link to 'Voir la fiche patient'.
- Statut:** 'Reflux gastrique' (red) and 'Régulier' (green).
- Facturation:** 'Statut au moment de la consultation' is 'Régulier' and 'Montant encaissé' is '25 DT'.
- Constantes vitales:**
 - TENSION: 132/86 mmHg
 - POULS: 73 bpm
 - TEMPÉRATURE: 37 °C
 - POIDS: 79 kg
- Informations cliniques:**
 - TYPE DE CONSULTATION: Suivi traitement
 - MOTIF DE CONSULTATION: Suivi reflux gastrique
 - OBSERVATIONS CLINIQUES: Reflux bien contrôlé par traitement.
 - ORDONNANCE / TRAITEMENT PRESCRIT: Oméprazole 40mg x1/jour le soir

Figure 23: Interface — Détail d'une consultation avec ordonnance

5.3.4 Tableau de bord

Le Tableau de bord affiche les statistiques clés via des **computed signals** Angular : total patients, répartition chroniques/non chroniques, revenus mensuels et cumulés, et les 5 dernières consultations. Le système d'alertes médicales détecte automatiquement les patients chroniques n'ayant pas eu de consultation depuis plus de 3 mois.

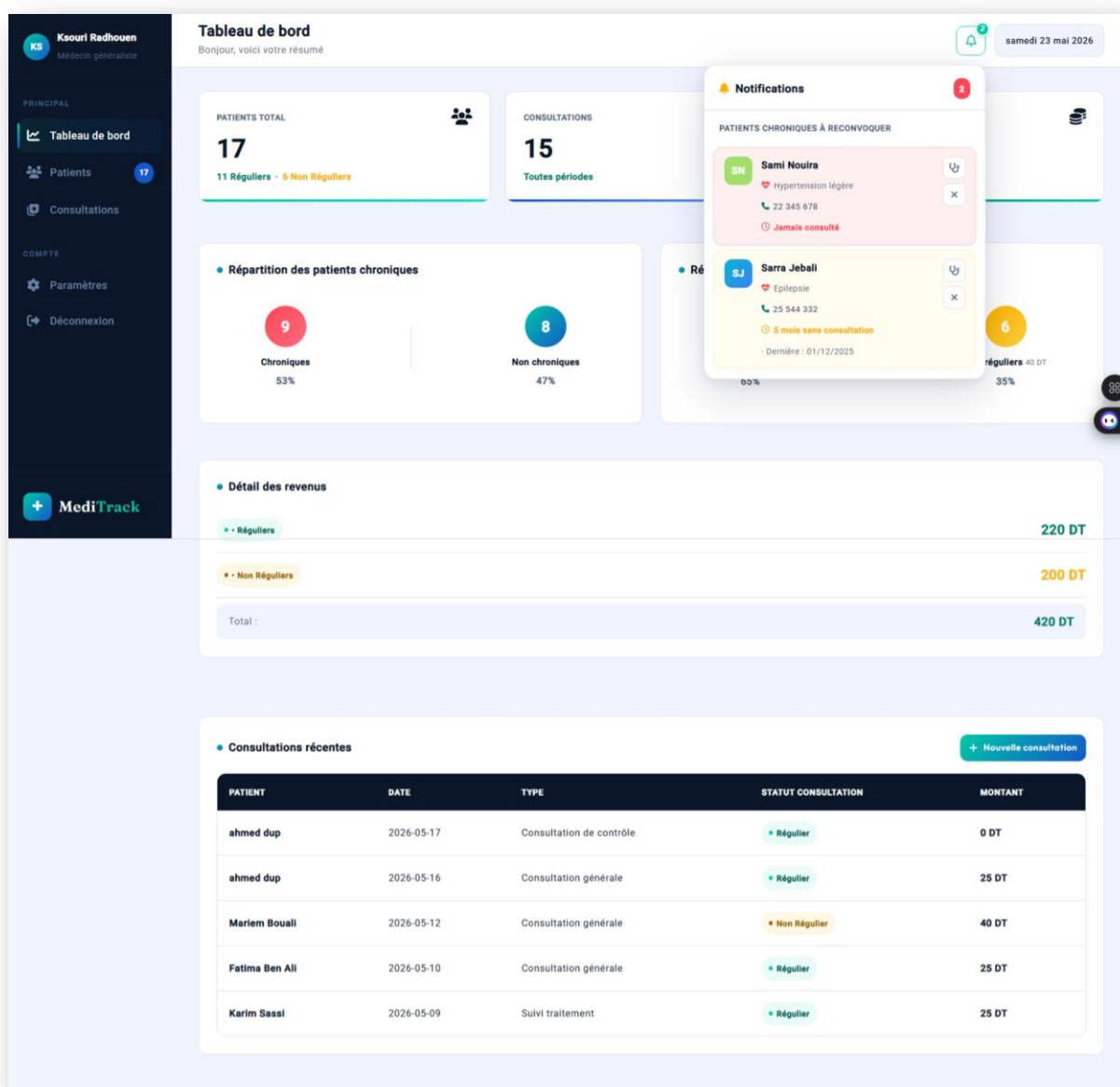


Figure 24: Interface — Tableau de bord avec statistiques et alertes (page web complète)

5.4 Itération 2 — Sprint 5 (27 avril 2026)

La deuxième itération a enrichi l'application avec des fonctionnalités avancées :

5.4.1 Gestion des comptes utilisateurs (Admin)

Le module utilisateurs permet à l'administrateur de créer, modifier et supprimer des comptes médecins. L'accès est protégé par l'adminGuard Angular, qui redirige automatiquement tout médecin tentant d'accéder à cette section.

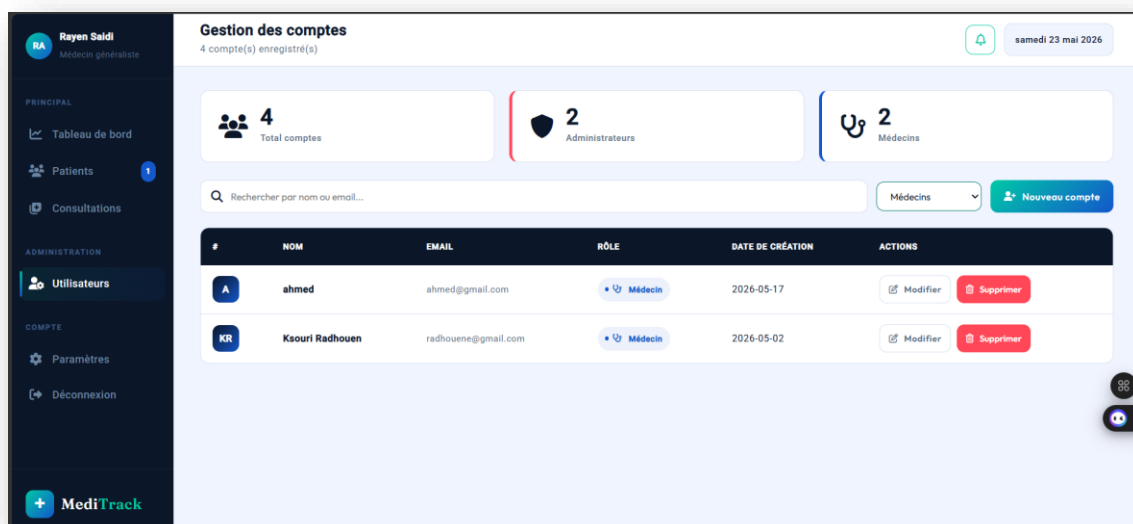


Figure 25: Interface — Liste des utilisateurs (Admin)

Nouveau compte
Créer un compte utilisateur

Créer un compte

Les champs marqués * sont obligatoires

Informations du compte

NOM COMPLET *
ex: Dr. Ahmed Ben Ali

ADRESSE EMAIL *
rayensold@gmail.com

RÔLE *
Médecin

Sécurité

MOT DE PASSE * (MINIMUM 4 CARACTÈRES)
.....

Moyen

Médecin
Accès aux patients et consultations. Pas d'accès à la gestion des comptes.

Annuler Créer le compte

Figure 26: Interface — Formulaire d'ajout d'un utilisateur

5.4.2 Module Paramètres

Le module de paramètres permet de configurer les tarifs de consultation (régulier / non régulier). Ces valeurs sont stockées en base de données dans la table **Paramètres** et récupérées au chargement du **DataService**, affectant automatiquement le calcul des montants de toutes les consultations futures.

Figure 27: paramètres

5.4.3 Système de notifications médicales

Le système de notifications médicales affiche dans la barre supérieure un badge avec le nombre d'alertes actives. Un computed signal dans le **DataService** calcule automatiquement la liste des patients chroniques dont la dernière consultation date de plus de 3 mois. Ces notifications peuvent être dismissées individuellement, avec persistance dans le **localStorage**.

Chapitre 6 : Tests et Validation

6.1 Stratégie de tests

Les tests ont été réalisés selon une approche progressive tout au long du développement, couvrant les aspects fonctionnels, d'interface, de performance et d'accessibilité.

6.2 Tests fonctionnels — Console Chrome (F12) + Postman

L'onglet Console des Chrome DevTools a été utilisé comme premier niveau de validation :

- Vérification des erreurs JavaScript au chargement des composants Angular
- Validation des appels API HTTP dans l'onglet Network (statuts 200, 201, 204, 404, 401)
- Inspection des réponses JSON retournées par les endpoints PHP
- Vérification des erreurs CORS (Cross-Origin Resource Sharing) et de leur résolution
- Validation du cycle complet : login → lecture des données → CRUD patients → consultations → logout

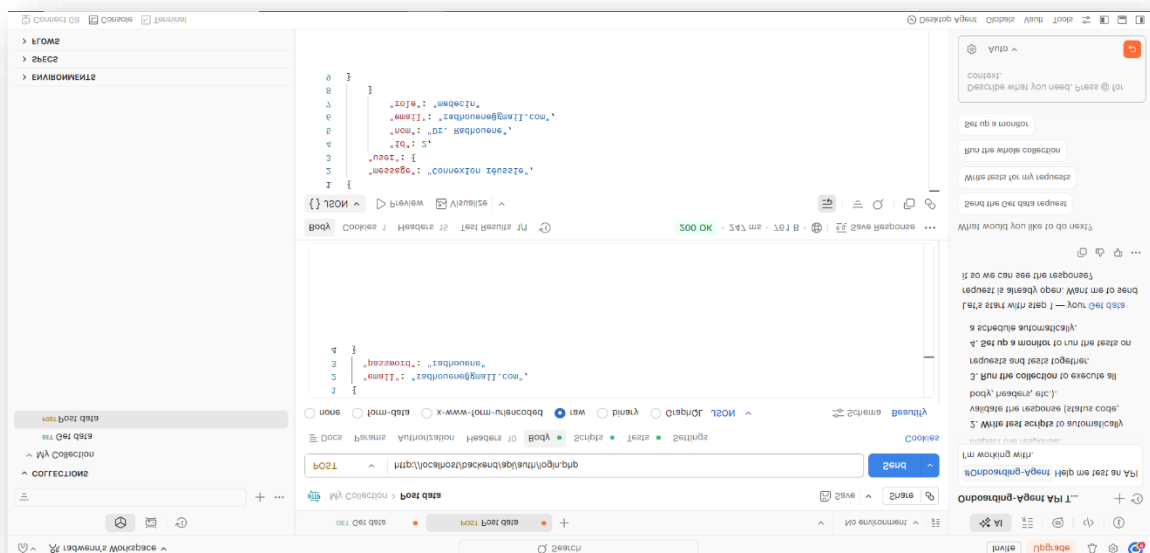


Figure 28: Postman

6.3 Tests de performance — Lighthouse

Lighthouse (intégré dans Chrome DevTools, onglet Lighthouse) a été utilisé pour mesurer les performances de l'application :

- **Performance** : évaluation du temps de chargement initial
- **Accessibilité** : détection des éléments sans attributs, contrastes insuffisants, labels manquants...
- **Bonnes pratiques** : vérification de l'utilisation de HTTPS, de l'absence de bibliothèques vulnérables

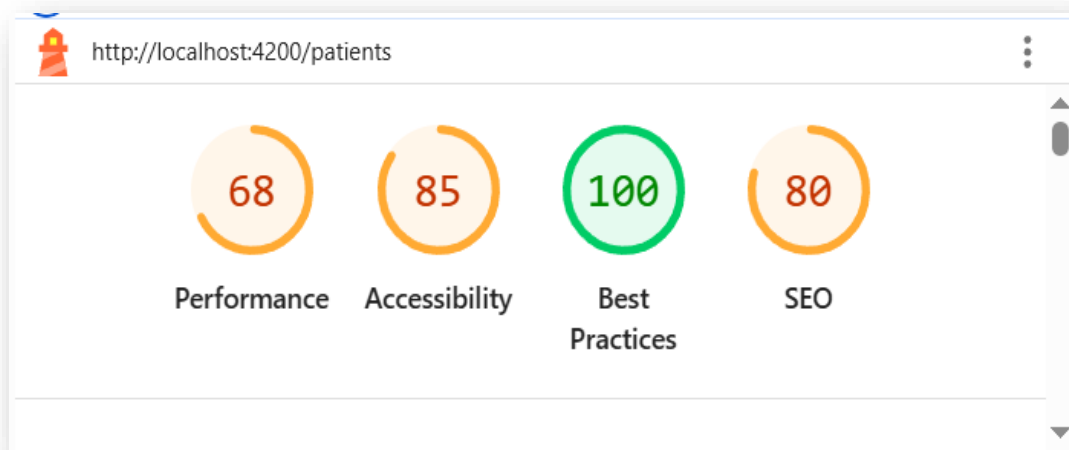


Figure 29: Lighthouse

6.4 Tests d'accessibilité

6.4.1 Axe DevTools (Extension Chrome)

Cette extension a été utilisée pour analyser automatiquement l'accessibilité de chaque page :

- Détection des violations WCAG 2.1 (manque de labels sur les champs de formulaire, rôles ARIA manquants)
- Vérification de la navigation au clavier (focus visible, ordre de tabulation logique)
- Identification des éléments interactifs sans texte alternatif.

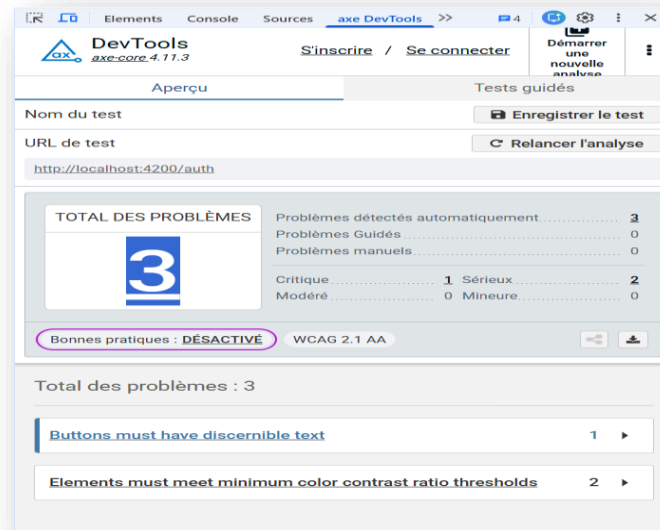


Figure 30: Axe DevTools (Extension Chrome)

6.4.2 Émulation d'appareils — Chrome DevTools

L'outil d'émulation d'appareils de Chrome DevTools (toggle device toolbar) a été utilisé pour tester le comportement responsive de l'application sur différentes tailles d'écran :

- Mobile
- Tablette
- Desktop standard

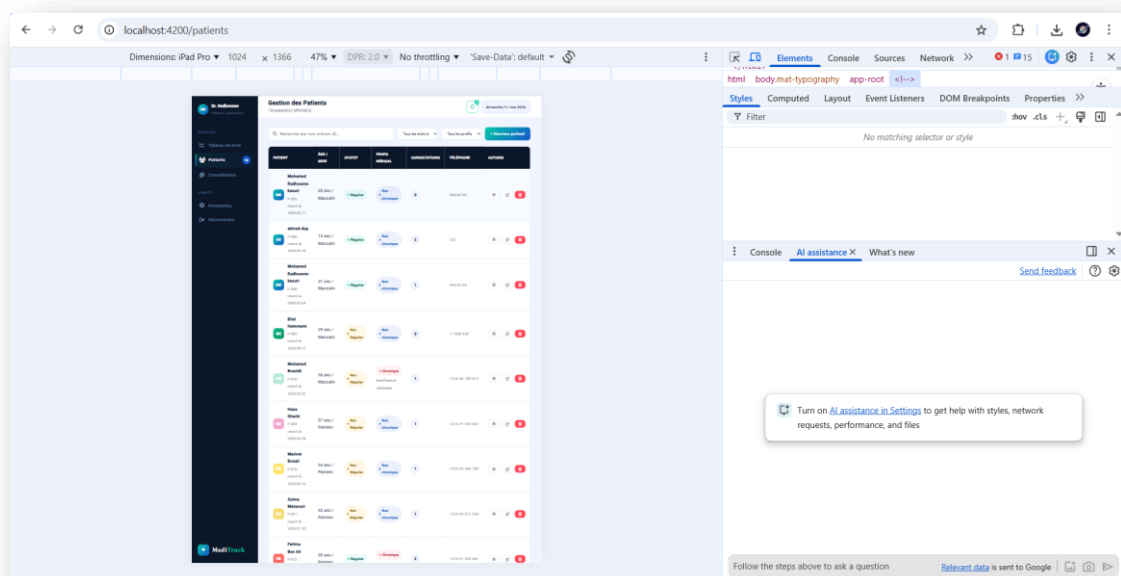


Figure 31: toggle device toolbar

6.5 Tableau récapitulatif des tests

Type de test	Outil utilisé	Résultat
Tests fonctionnels API	Postman + Console Chrome	Tous les endpoints validés
Tests de navigation	Chrome DevTools (Network)	Routing Angular fonctionnel
Tests de performance	Lighthouse (Chrome)	Scores satisfaisants
Tests d'accessibilité	Axe DevTools (Extension)	Violations identifiées et corrigées
Tests responsives	Chrome DevTools (Émulation)	Mobile, tablette, desktop OK

Tableau 8: Tableau récapitulatif des tests

Conclusion et Perspectives

7.1 Bilan du projet

Le projet **MediTrack** a été mené à terme avec succès. L'ensemble des User Stories prévues au Product Backlog ont été implémentées et validées. L'application permet à un médecin de gérer ses patients et ses consultations de manière sécurisée, avec un tableau de bord réactif et un système d'alertes médicales automatisées.

Sur le plan technique, ce projet nous a permis de maîtriser une stack moderne (Angular 18 Signals + PHP REST API + MySQL), d'appliquer des patterns architecturaux concrets (Singleton, Guard, Lazy Loading, Repository Pattern), et d'utiliser des outils professionnels (Git, Postman, Lighthouse, Axe DevTools).

Sur le plan méthodologique, la démarche Scrum a structuré notre travail en sprints progressifs, chaque itération apportant des améliorations basées sur les retours du professeur. Cette approche nous a appris l'importance de la conception avant l'implémentation et de la documentation continue.

7.2 Perspectives d'amélioration

- Intégration d'un module de génération automatique d'ordonnances (PDF)
- Module de prise de rendez-vous en ligne
- Tableau de bord avec graphiques interactifs (Chart.js ou D3.js)
- Application mobile (Angular PWA ou React Native)
- Déploiement sur serveur cloud (Heroku, Railway, ou VPS)
- Module de messagerie interne entre médecin et patient
- Intégration d'un module d'intelligence artificielle pour l'aide au diagnostic (comme prévu initialement au cahier des charges V1, non retenu en V3 faute de temps)

Webographie

Documentation officielle

- [1] Angular — Documentation officielle : <https://angular.dev> (consulté en 2025–2026)
- [2] Angular Material Components : <https://material.angular.io>
- [3] PHP — Manuel officiel : <https://www.php.net/manual/fr/>
- [4] MySQL — Documentation : <https://dev.mysql.com/doc/>
- [5] PDO — Documentation PHP : <https://www.php.net/manual/fr/book.pdo.php>

Outils de test et qualité

- [6] Google Lighthouse : <https://developer.chrome.com/docs/lighthouse/>
- [7] Axe DevTools — Extension Chrome : <https://www.deque.com/axe/devtools/>
- [8] WebAIM Contrast Checker : <https://webaim.org/resources/contrastchecker/>
- [9] Chrome DevTools — Documentation : <https://developer.chrome.com/docs/devtools/>

Méthodologie et conception

- [10] Scrum Guide officiel : <https://scrumguides.org/scrum-guide.html>
- [11] Karma + Jasmine (tests unitaires Angular) : <https://karma-runner.github.io>
- [12] Git — Documentation officielle : <https://git-scm.com/doc>